IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received May 17, 2018, accepted June 18, 2018, date of publication June 22, 2018, date of current version July 12, 2018.

*Digital Object Identifier 10.1109/ACCESS.2018.2849821*

# Formal Specification and Verification of Self-Adaptive Concurrent Systems

**MUHAMMAD ILYAS FAKHIR AND SYED ASAD RAZA KAZMI**
Department of Computer Science, Government College University, Lahore 54000, Pakistan
Corresponding author: Muhammad Ilyas Fakhir (fakhir@gcu.edu.pk)

**ABSTRACT** The assurance of required quality properties is one of the major challenges in self-adaptive systems (SASs). SASs have the capability to adapt their dynamic behavior autonomously at runtime due to uncertain changes in the environment. In general, an SAS is much difficult to specify and verify, because of its highly complex internal behavior and especially when time constraints are involved. In this research, we use modal $\mu$-calculus ($\mathcal{M}_\mu$) for the specification and verification of colored Petri nets-based self-adaptive concurrent systems. We propose self-adaptive multi-agent concurrent system (SMACS) framework that is specifically designed for complex architectures. The internal structure of SMACS framework is based on MAPE-K feedback loop. Each phase of the feedback loop works as an internal agent (Int-Agent) known as Monitor Int-Agent, Analyzer Int-Agent, Planer Int-Agent, and Executer Int-Agent. The decentralized approach is being used in this research, and due to this approach, all agents intelligently adapt their behavior in the environment and send updates to other agents. For verification of internal properties like liveness, safeness, and deadlock-freedom of each agent, the TAPA model checker is being used. For the implementation of SMACS framework, traffic monitoring system is chosen as a case study.

**INDEX TERMS** Self-adaptive, multi-agent, formal specification, formal verification, modal $\mu$-calculus, TAPA model checker.

## I. INTRODUCTION

Formal specification and verification is one of the most prominent and challenging fields of theoretical computer science, while the underlying objective of this area is to provide formal methods in order to assure the correctness in accordance with the given specifications. For example, some safety critical systems or programs like autopilot that assists the pilot during the flight, the software for managing nuclear power plant controllers, or other safety critical systems don't exhibit faulty behavior. (The probable malfunctioning of software running on such systems is addressed using formal methods.) In order to avoid any kind of error in such safety critical systems, the formal methods are the best choice for verification of the behavior of the systems. Primarily, these systems are specified in any of the formal specification languages and the corresponding model checking techniques are used for the verification.

First of all formal modeling techniques are used to model a system and then for verification of its specific behavior any formal language is applied in conjunction with corresponding model checker. For the specification of such complex systems, temporal logics based formal languages are used and for the verification of their correctness, the most expressive technique is model checking [1]–[3], but model checking has the state space explosion problem for complex systems. Bounded model checking is proposed to handle state space explosion problem [4] to some extent; however, it does not provide complete solution to the problem. Similarly, by the use of some other approaches an infinite behavior model is transformed into finite model like; as state-transition graph or coverability graph, the modeling technique used in Petri nets. A brief introduction to modeling and analyzing complex systems is presented in [5] as well as complex systems with self-adaptivity is also presented in [6].

We propose a framework of Self-Adaptive Multi-Agent Concurrent System (SMACS) which primarily deals with complex systems. In this research work Colored Petri Net (CPN) is being used to model such framework [7] and modal $\mu$-calculus is being used for specification and verification of internal properties of the system. Petri net is a best modeling approach to model multi-agent based concurrent systems. However, it is difficult to model the internal behavior of self-adaptive agent; to overcome this exertion modal $\mu$-calculus has been used. To verify properties

written in modal $\mu$-calculus its corresponding model checker TAPA is available [8]. Modal $\mu$-calculus is a dominant fixed point-based temporal logic [9]. Although, Intuitionistic Linear Time $\mu$-Calculus (I$\mu$TL) is more expressive than modal $\mu$-calculus [10]; however, its model checker is not available. A CPN based concurrent behavior model is proposed in [11], in which safety and bounded properties are verified through I$\mu$TL theoretically. The proposed SMACS framework is a MAPE-K based self-adaptive system and each phase of MAPE-K feedback loop works as an adaptive agent, known as internal agent. The idea of internal agents is deeply discussed in ARTIS agent architecture, which is a hard real-time multi-agent system for hard real-time environment [12]. In this architecture, every ARTIS agent has multiple internal agents and every internal agent has ability to solve particular problem. MAPE-K is a well-known engineering approach to realize self-adaptation [13]. The self-adaptive agent having the capability to interact or extract due to any abrupt change in the environment. Each agent cannot learn only from other agents, but also from its own experience during functioning in the environment. Self-adaptivity is considerably prominent research area in formal methods, and a lot of work is carried out for modeling and verification of multi-agent based architectures in the last few decades. While some work has been done in the formal modeling and analyzing MAPE-K feedback loop for self-adaptation as presented in [14] and [15], having a limited scope as compared to our work. Similarly, according to our best of knowledge two approaches; timed automata in [16] and Z method in [17] are used for the formal specification and verification of decentralized self-adaptive system through feedback loop. MAPE-K feedback loop is also used to model traffic monitoring system by applying inter-loop coordination and intra-loop coordination phenomena in [18]. ActivFORMS [19] approach is recently proposed to guarantee adaptation goals at the design time and also at implementation time. In this framework a MAPE-K based formal model is executed at runtime by a virtual machine and a dynamic change occurs after changing internal goals.

For the modeling of self-adaptive systems two types of approaches have been used; top-down and bottom-up approaches. In the former case self-adaptive systems, the control is centralized and behave with guidance of central controller, which evaluates its surroundings behavior and adapts itself accordingly. On the other hand, bottom-up self-adaptive systems have decentralized phenomena, such type of systems have self-adaptive cooperation or self-organization as discussed in [20]. In the bottom-up approach, the structure of the system is just like multi-agent system, because in MAS each agent updates its own behavior and also updates the environment which guides the other agents. The key features of multi-agent systems in the engineering of self-adaptive systems are specifically; loose coupling, context sensitivity, robustness in response to failure and unexpected concurrent events. To model an agent based system a novel VOMAS (Virtual Overlay Multi-agent System) approach is used to validate simulation of multi-agent systems [21]. For adaptation purposes, transition based or automata based architectures have been proposed such as; synchronous adaptive systems of MARS [22] and S[B] systems [23]. This work is based on the multi-level view of self-adaptivity, where the actual dynamic behavior is described in lower behavioral level and dynamic change in environmental level is described in upper structural level. A runtime approach for modeling and analysis of dynamic adaptive system is also proposed in [24]. In this research middle-ware approach is presented to address dynamic reconfiguration during execution. For formal modeling of multi-agent system object-Z language is also used to model agent's decision-making procedures and inter-agent negotiation mechanisms [25]. For distributed multi-agent system with time constraint using TCOZ language is proposed in [26]. TCOZ is also used in [27] for modeling real-time self-adaptive multi-agent system. The multi-agent system is formally modeled and analyzed using mu-calculus and Timed-Arc Petri net in [28] and [29]. Similarly, in case of concurrency, the well known approaches like CCS, CSP and ASP have been used to model self-adaptive system as a subclass of reactive system as proposed in [30].

For the verification of such complex systems, different model checking approaches have been applied. Goal-level model checking analysis of adaptive system is discussed in SOTA approach [31] where a SOTA language is used to dynamically describe dependencies among components. In SOTA framework [32] decentralized feedback loop based approach is used and for the modeling and analysis of self-adaptive system, SimSOTA simulator is used. A three layered separation formalism among managed components, system components and context entities is proposed in [33] to model context-aware systems and for model checking Maude model checker is used. Maude is a reflective logical language, that is also applied in [34] to model well-engineered self-adaptive robotic components. Abstract State Machines (ASM) formal method is also used to model self-adaptive systems as discussed in [35]. To model feedback loop for self-adaptive systems the above mentioned techniques do not support deeply. However, our proposed framework is a MAPE-K based self-adaptive multi-agent system having concurrent behavior and according to our point of view no work has been done in this specific research area. In our approach we explicitly explore MAPE-K feedback loop to recognize and ratify self-adaptivity by using formal approaches. Safety, liveness and deadlock freedom properties of SMACS framework are then verified through the TAPA model checker.

The remaining paper is organized as: in section 2 preliminaries for proposed modeling techniques are discussed, in section 3 proposed model is discussed, in section 4 a case study for the verification of SMACS framework is discussed and finally conclusion and future work is discussed in section 5.

## II. PRELIMINARIES

Self-adaptive systems having dynamic concurrent behavior, according to which systems adapt behavior dynamically as

presented in [36]. Modeling of systems in Colored Petri Net is a suitable approach, because CPN has flexibility to model discrete event dynamic system and to attain true concurrency during execution of such complex model is also possible. However, to model the internal behavior of self-adaptive system, a much expressive formal language is needed, due to this reason modal $\mu$-calculus is proposed for formal specification and verification. In this section, we will discuss terminologies use to help for formal modeling, specification and analysis of self-adaptive multi-agent system.

### A. SELF-ADAPTIVE SYSTEM

The Self-Adaptive Systems having ability to adapt behavior by sensing any change in its environment. It is the behavior of SAS to organize itself autonomously by accommodating changes in its environment and context. Some SASs may have ability to adapt changes in an environment without human interaction through some higher-level policies as presented in [13]. Just like agents in MAS, adaptation is a process of switching between models in the multi-model system. So it is the capability of a system to adapt the behavior of the environment by selectively switching and executing between models. For modeling and deploying high-level complex system so that human interaction can be reduced, is a key feature of self-adaptivity.

The adaptation mechanism is decomposed by Selehie and Tahvildari in 2011 into several processes: *monitoring* the environment and software objects (i.e. context-awareness & self-awareness), *analyzing* substantial variations, *planning* of reaction, and *executing* the decisions. For the specification of SASs, monitoring and switching between adaptive behaviors, goal-oriented models have proven their effectiveness. The ability to reason about partial goal satisfaction is a strength of goal-oriented modeling.

### B. COLORED PETRI NETS

In Colored Petri nets, tokens are in the form of colors denoting specific data types. Any transition is called enabled for firing when all of its input places having valid colored tokens, and valid arc bindings are used to produce the necessary colored tokens to its output places. The firing of transitions in CPN can modify the data values of these colored tokens.

The Formal definition of CPN is as follows:

CPN $= (\Sigma, P, T, A, V, C, G, E, I)$, where:

1) $\Sigma$ is a finite set of non-empty color sets.
2) $P$ is a finite set of places.
3) $T$ is a finite set of transitions.
4) $A$ is a set of directed arcs such that: $A \subseteq P \times T \cup T \times P$.
5) $V$ is a typed finite variables set such that: $Type[v] \in \Sigma, \forall v \in V$.
6) $C$ is a color function (non-empty color set). It is defined as $C : P \rightarrow \Sigma$.
7) $G$ is a guard function defined as $G : T \rightarrow EXPR_v$. It assigns a guard to each transition $t$ such that: $\forall t \in T : [Type(G(t)) = Bool]$.
8) $E$ is an arc expression function, defined as: $E : T \rightarrow EXPR_v$ such that: $\forall a \in A : [Type(E(a)) = C(p)_{MS}]$, where $p$ is the place connected to arc $a$.
9) $I$ is an initialization function, defined as: $I : P \rightarrow EXPR_\phi$ such that: $\forall p \in P : [Type(I(p)) = C(p)_{MS}]$.

### C. THE MODAL $\mu$-CALCULUS ($\mathcal{M}_\mu$)

The modal $\mu$-calculus provides *least fixed-point* operator "$\mu$" and *greatest fixed-point* operator "$\nu$", which provide external fixed-point characterization of correctness properties. Intuitively, $\mu$-calculus having capability to express the modalities into recursive patterns. E.g. reachability can be expressible: "$a$ will be eventually true along all paths" can be expressed in $\mathcal{M}_\mu$ as $\mu\mathcal{Z}_\mu.(a \vee \langle\mathcal{Z}_\mu\rangle)$, the least fixed-point of $a \vee \langle\mathcal{Z}_\mu\rangle$ where $\mathcal{Z}_\mu$ is set of states. Similarly a nonterminating state of a system can be expressed as $\nu\mathcal{Z}_\mu.(\langle\mathcal{Z}_\mu\rangle)$, the greatest fixed-point of $\langle\mathcal{Z}_\mu\rangle$. Moreover, *bisimulation* is a classical relation linked with $\mathcal{M}_\mu$. Bisimulation is actually a behavioral equivalence between systems and it has an origin in concurrency. It is studied to identify the formulae in first-order logic, which are equivalent to modal logic formulae. Originally bisimulation is defined as a fixed point of a monotone self-map complete lattice. Two systems are bisimilar behaviorally if their specification written in $\mathcal{M}_\mu$ cannot be distinguished. Due to this phenomena $\mathcal{M}_\mu$ is an invariant under bisimulation. This characteristic of $\mathcal{M}_\mu$ will be helpful for writing the specification of concurrent behavior of our proposed framework. Modalities are expressed by transitions, as formula $\langle\beta\rangle\alpha$ holds if there is an outgoing $\beta$-transition to some states satisfying $\alpha$, similarly formula $[\beta]\alpha$ holds if there are all outgoing $\beta$-transition states satisfying $\alpha$.

#### 1) SYNTAX OF $\mathcal{M}_\mu$

$$\begin{aligned}\mathcal{F}_\mu ::= {} & \mathcal{Z}_\mu | \top_\mu | \bot_\mu | \neg\mathcal{F}_\mu | \mathcal{F}_{\mu 1} \wedge \mathcal{F}_{\mu 2} | \mathcal{F}_{\mu 1} \vee \mathcal{F}_{\mu 2} | \\ & \mathcal{F}_{\mu 1} \rightarrow \mathcal{F}_{\mu 2} | \langle\beta\rangle\mathcal{F}_{\mu 1} | [\beta]\mathcal{F}_{\mu 2} | \mu\mathcal{Z}_\mu.\mathcal{F}_\mu | \nu\mathcal{Z}_\mu.\mathcal{F}_\mu\end{aligned}$$

#### 2) SEMANTICS OF $\mathcal{M}_\mu$

Let $\mathcal{N}$ be a label transition system, which represents a Petri Net (PN) Model, and $\delta : \mathcal{Z}_\mu \rightarrow 2^{\mathcal{S}}$ is a firing sequence in the environment ($\mathcal{Z}_\mu \subseteq \mathcal{S}$).

$$\begin{aligned}[\![\top_\mu]\!]^\delta_\mathcal{N} &= \mathcal{S} \quad [\![\bot_\mu]\!]^\delta_\mathcal{N} = \phi \\ [\![\mathcal{Z}_\mu]\!]^\delta_\mathcal{N} &= \delta(\mathcal{Z}_\mu) \\ [\![\neg\mathcal{F}_\mu]\!]^\delta_\mathcal{N} &= \mathcal{S} \backslash [\![\mathcal{F}_\mu]\!]^\delta_\mathcal{N} \\ [\![\mathcal{F}_{\mu 1} \wedge \mathcal{F}_{\mu 2}]\!]^\delta_\mathcal{N} &= [\![\mathcal{F}_{\mu 1}]\!]^\delta_\mathcal{N} \cap [\![\mathcal{F}_{\mu 2}]\!]^\delta_\mathcal{N} \\ [\![\mathcal{F}_{\mu 1} \vee \mathcal{F}_{\mu 2}]\!]^\delta_\mathcal{N} &= [\![\mathcal{F}_{\mu 1}]\!]^\delta_\mathcal{N} \cup [\![\mathcal{F}_{\mu 2}]\!]^\delta_\mathcal{N} \\ [\![\mathcal{F}_{\mu 1} \rightarrow \mathcal{F}_{\mu 2}]\!]^\delta_\mathcal{N} &= (\mathcal{S} \backslash [\![\mathcal{F}_{\mu 1}]\!]^\delta_\mathcal{N}) \cup [\![\mathcal{F}_{\mu 2}]\!]^\delta_\mathcal{N} \\ [\![\langle\beta\rangle\mathcal{F}_\mu]\!]^\delta_\mathcal{N} &= \{s \in \mathcal{S} \mid \exists_{t \in \mathcal{S}, \sigma \in [\![\beta]\!]} s \xrightarrow{\sigma} t \wedge t \in [\![\mathcal{F}_\mu]\!]^\delta_\mathcal{N}\} \\ [\![[\beta]\mathcal{F}_\mu]\!]^\delta_\mathcal{N} &= \{s \in \mathcal{S} \mid \exists_{t \in \mathcal{S}, \sigma \in [\![\beta]\!]} s \xrightarrow{\sigma} t \Rightarrow t \in [\![\mathcal{F}_\mu]\!]^\delta_\mathcal{N}\} \\ [\![\mu\mathcal{Z}_\mu.\mathcal{F}_\mu]\!]^\delta_\mathcal{N} &= \mu\mathcal{T}_\mu \subseteq \mathcal{S}.\varphi_\delta(\mathcal{T}_\mu) \\ [\![\nu\mathcal{Z}_\mu.\mathcal{F}_\mu]\!]^\delta_\mathcal{N} &= \nu\mathcal{T}_\mu \subseteq \mathcal{S}.\varphi_\delta(\mathcal{T}_\mu)\end{aligned}$$

Where $\varphi_\delta : 2^{\mathcal{S}} \rightarrow 2^{\mathcal{S}}$ is a function defined as $\varphi_\delta(\mathcal{T}_\mu) = [\![\mathcal{F}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\mathcal{T}_\mu]}$.

The environment is omitted from $\varphi$, if $\mathcal{F}_\mu$ is closed. Since $\varphi$ is monotonic function and $\mathcal{S}$ is finite, so we use an iterative function to find fixed points. i.e. $\varphi^0(\mathcal{T}_\mu) = \mathcal{T}_\mu$, and $\varphi^{k+1}(\mathcal{T}_\mu) = \varphi(\varphi^k(\mathcal{T}_\mu))$, then redefining of fixed-points are as follow:

$$[\![\mu\mathcal{Z}_\mu.\mathcal{F}_\mu]\!]_{\mathcal{N}}^{\delta} = \bigcup_i \varphi_\delta^i(\phi) \;\&\; [\![\nu\mathcal{Z}_\mu.\mathcal{F}_\mu]\!]_{\mathcal{N}}^{\delta} = \bigcap_i \varphi_\delta^i(\mathcal{S})$$

In fact, for both cases it suffices to start computing with $\varphi^0$ and apply function recursively until computing a stable result. So with the help of such result, we will compute some places (states) of PN model in which a $\mu$-calculus formula holds. We will show by using the formula $\{\mu, \nu\}\mathcal{Z}_\mu.[t_i]false \vee \langle true\rangle\mathcal{Z}_\mu$, where the least or the greatest fixed-point hold for the PN model in Figure 1. Let $\varphi$ be a function s.t. $\varphi_\delta(\mathcal{T}_\mu) = [\![[t_i]false \vee \langle true\rangle\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\mathcal{T}_\mu]}$. First we will solve this net for finding least fixed-point and will start with approximation at $\phi$:

$$\begin{aligned}
\varphi^0(\phi) &= \phi\\
\varphi^1(\phi) &= [\![[t_i]false \vee \langle true\rangle\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\phi]}\\
&= [\![[t_i]false]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\phi]} \cup [\![\langle true\rangle\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\phi]}\\
&= \{s_i \in \mathcal{S} | \exists_{s_j\in\mathcal{S},\sigma\in\{\prod_i t_i\}} s_i \xrightarrow{\sigma} s_j \Rightarrow s_j \in \phi\} \cup \{s_i\\
&\in \mathcal{S} | \exists_{s_j\in\mathcal{S},\sigma\in\{\prod_i t_i\}} s_i \xrightarrow{\sigma} s_j \Rightarrow s_j \in [\![\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\phi]}\}\\
&= \{s_2\} \cup \phi\\
&= \{s_2\}\\
\varphi^2(\phi) &= [\![[t_i]false \vee \langle true\rangle\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\{s_2\}]}\\
&= [\![[t_i]false]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\{s_2\}]} \cup [\![\langle true\rangle\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\{s_2\}]}\\
&= \{s_2\} \cup \{s_i \in \mathcal{S} | \exists_{s_j\in\mathcal{S},\sigma\in\{\prod_i t_i\}} s_i \xrightarrow{\sigma} s_j \Rightarrow s_j\\
&\in [\![\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\{s_2\}]}\}\\
&= \{s_2\} \cup \{s_0\}\\
&= \{s_0, s_2\}\\
\varphi^3(\phi) &= [\![[t_i]false \vee \langle true\rangle\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\{s_0,s_2\}]}\\
&= [\![[t_i]false]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\{s_0,s_2\}]} \cup [\![\langle true\rangle\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\{s_0,s_2\}]}\\
&= \{s_2\} \cup \{s_i \in \mathcal{S} | \exists_{s_j\in\mathcal{S},\sigma\in\{\prod_i t_i\}} s_i \xrightarrow{\sigma} s_j \Rightarrow s_j\\
&\in [\![\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\{s_0,s_2\}]}\}\\
&= \{s_2\} \cup \{s_0\}\\
&= \{s_0, s_2\}
\end{aligned}$$

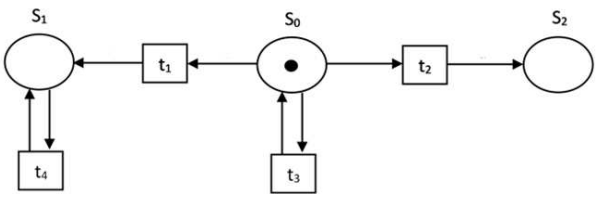



**FIGURE 1.** A PN model.

So the property does not hold in $s_0$ and $s_2$. Hence $\{s_0, s_2\}$ is the least fixed-point set and the result shows that the deadlock will occur during the execution of the system from $s_0$ to $s_2$. This shows that liveness property does not hold at $s_2$. Now for greatest fixed-point:

$$\begin{aligned}
\varphi^0(\mathcal{S}) &= \mathcal{S}\\
\varphi^1(\mathcal{S}) &= [\![[t_i]false \vee \langle true\rangle\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\mathcal{S}]}\\
&= [\![[t_i]false]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\mathcal{S}]} \cup [\![\langle true\rangle\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\mathcal{S}]}\\
&= \{s_i \in \mathcal{S} | \exists_{s_j\in\mathcal{S},\sigma\in\{\prod_i t_i\}} s_i \xrightarrow{\sigma} s_j \Rightarrow s_j \in \phi\}\\
&\quad \cup \{s_i \in \mathcal{S} | \exists_{s_j\in\mathcal{S},\sigma\in\{\prod_i t_i\}} s_i \xrightarrow{\sigma} s_j \Rightarrow s_j \in [\![\mathcal{Z}_\mu]\!]_{\mathcal{N}}^{\delta[\mathcal{Z}_\mu:=\mathcal{S}]}\}\\
&= \{s_2\} \cup \{s_0, s_1\}\\
&= \{s_0, s_1, s_2\}\\
&= \mathcal{S}
\end{aligned}$$

So at all places, greatest fixed-point property is satisfied, to show that system remains in safe condition.

## III. SELF-ADAPTIVE MULTI-AGENT CONCURRENT SYSTEM FRAMEWORK (SMACS)

Figure 3 represents a framework which is suitable for modeling systems having complex internal behavior. Formal modeling techniques are being used to model concurrent systems for many decades and productive results are being achieved by the researchers. In the proposed work, agents can perform multiple high-level tasks concurrently to complete the goal derives for the system or agent in a certain time span. These high-level concurrent tasks can be generated either by internal communication of agents or by sensing new jobs in the environment. An agent is self-directed, intelligent and cooperative enough to collaborate with other agents to perform tasks in a highly complicated environment. In this formal specification framework, we are using use bottom-up approach for achieving true concurrency, as it is the best approach for modeling multi-agent based systems to cover important concurrent aspects of agents. As discussed in ARTIS agent architecture [12], an agent has multiple internal agents and every internal agent has ability to solve a particular problem. When sensor channel observes changes in the environment, it intelligently notifies to MAPE-K phases also known as internal-agents (Int-Agents) to complete the task. Each agent is composed of MAPE-K based internal architecture. MAPE agents complete the request based on their knowledge and send back their decision to the environment through reactor channel. These internal-agents are directly connected to knowledge based system updates for sending and receiving updates. Based on the results forwarded from an upper layer, these agents act to complete the task and send updates to the environment for other agents. The Monitor Int-Agent also gets updates through the managed system, and after completing a task the Executer Int-Agent also forwards result to the managed system as well. The managed system is responsible to control all hardware devices attached with it and also updates itself within a specific time span.

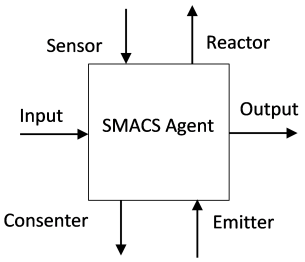



**FIGURE 2.** The structure of SMACS agent.

A SMACS agent can be described as a tuple $\langle \mathcal{I}, \mathcal{O}, \mathcal{S}, \mathcal{R}, \mathcal{E}, \mathcal{C} \rangle$ (Figure 2):

- $\mathcal{I}$: is a set of inputs which satisfy post-conditions, i.e. $\{i|post(i) \in \mathcal{I}\}$
- $\mathcal{O}$: is a set of outputs which satisfy pre-conditions, i.e. $\{o|pre(o) \in \mathcal{O}\}$
- $\mathcal{S}$: is a set of sensed jobs exist in the environment and forward to Monitor Int-Agent by Sensor channel, i.e. $\{\sum_{i=1}^{n} j_i \in \mathcal{S} \wedge \mathcal{S} \subset \mathcal{I}\}$
- $\mathcal{R}$: is a set of all completed job forwarded from Executer Int-Agent to Reactor channel, i.e. $\{\sum_{o=1}^{n} j_o \in \mathcal{R} \wedge \mathcal{R} \subset \mathcal{O}\}$
- $\mathcal{E}$: (Emitter) is a set of information "requests or updates" generated by an "internal agent" of an agent to an "internal agent" of all other agents through "knowledge based system updates" state. i.e. "Monitor Int-Agent" of Agent-i wants to share information with "Monitor Int-Agent" of Agent-j, then it will emit updates or requests to communicate.
- $\mathcal{C}$ : (Consenter) is a set of acknowledgements of all requests generated by internal agents.

### A. MODELING OF SMACS FRAMEWORK

The SMACS framework is modeled in CPN as shown in Figure 4. In Petri net architecture places are passive entities and transitions are active entities, so by this phenomenon all internal agents are planned as transitions. The environment of SMACS framework is composed of processes and attributes. The observer observes the attribute's value in the environment, and when a new process is initiated in the environment attribute's value may be modified. The environment has the capability to update itself after a specific time. After these updated information, new modernized attributes are available at output channel know as a sensor. The sensor is a channel used for communication from environment to Monitor Int-Agent, similarly reactor is also channel used for communication from Execute Int-Agent to the environment. The sensor is responsible to perform jobs like: waiting, sensing, filtering and notifying. It initiates the process of sensing and this process is activated in the managed system to apply adaptation actions after notifying to Monitor Int-Agent through Process-ID. After sensing the new job from environment, filtering phenomenon based on threshold (minimum or maximum variation of data) value is applied. Filtering is a domain specific criteria for each case. Monitor performs the jobs like verification of preprocessed data based on threshold value and updating of data for Analyzer Int-Agent. Monitor continuously checks the status of sensor channel for new incoming processes. A Monitor-ID is assigned to all new

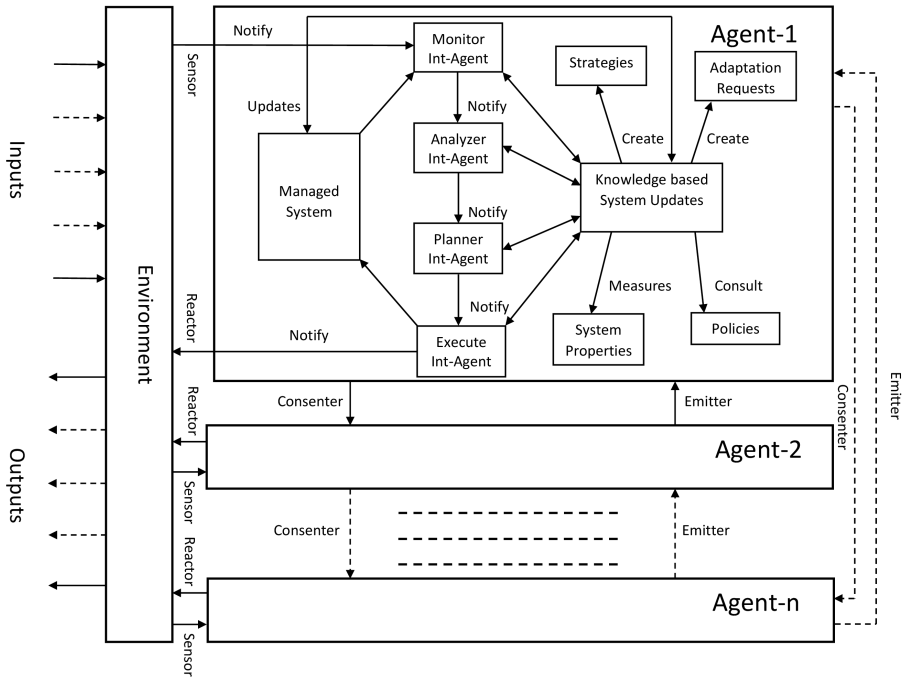



**FIGURE 3.** SMACS framework.

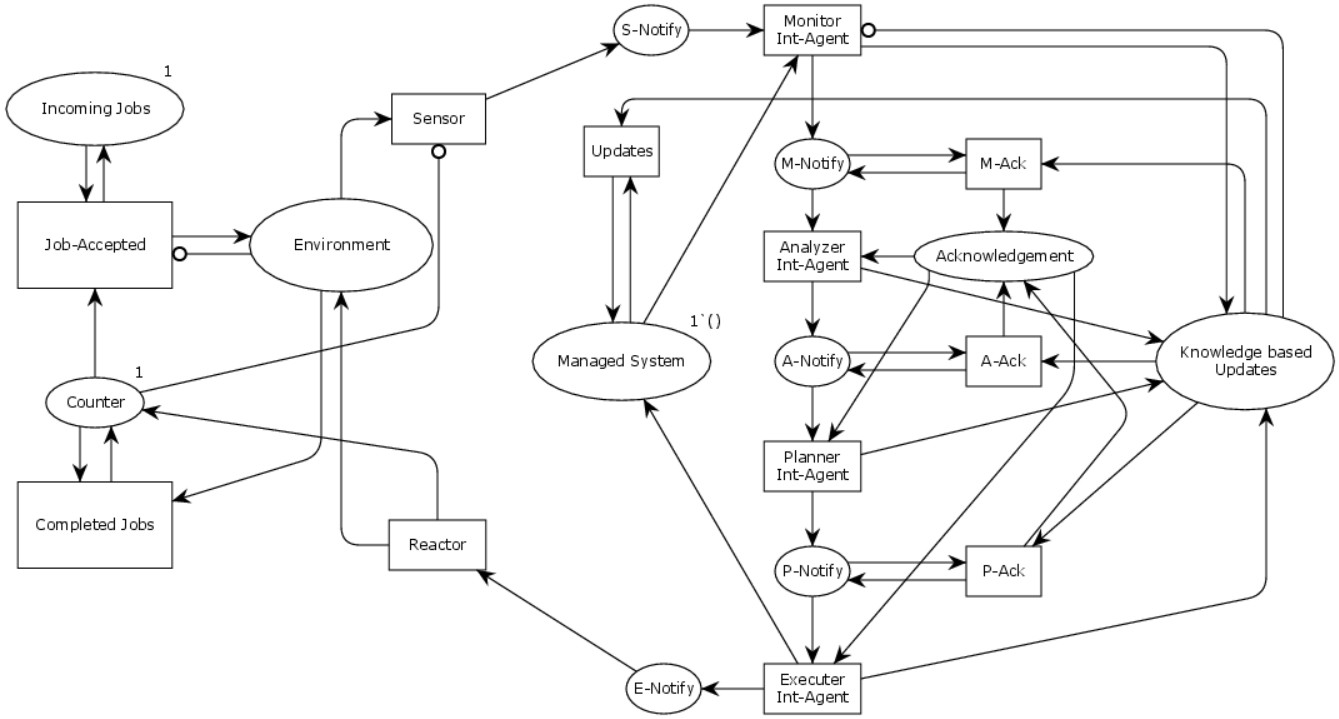



**FIGURE 4.** CPN model of SMACS framework.

coming processes. At this phase system property is applied to new data in its adaptation process and investigating monitoring activities. After this Analyzer Int-Agent receive data from Monitor Int-Agent and performs action on the basis of existing data repository. This phase adds new symptoms to the data repository with the help of existing information and finally adaptation request is forwarded to the next phase. This phase is also responsible to create new adaptation requests. The Planner Int-Agent phase is responsible to satisfy results forwarded through previous phase in managing system and then send forward to Executer Int-Agent. Due to unsatisfactory results this phase is responsible to add new resources and new plan to update results in the knowledge based database. This phase is also responsible to create new strategies and consult new policies. Executer Int-Agent takes actions against updated plan and these actions can be executed either in sequential order or concurrent order or may be in hybrid format. This phase forward updates to Reactor channel which is responsible to apply changes to the system or the environment. The managed system also updates itself after receiving updated data from Executer Int-agent. After taking some counteractive actions, this phase will put system to some desirable or adequate form. Reactor can set updated reading and set updated threshold and finally release the job to the environment.In a multi-agent environment, each internal agent of Agent-1 is directly connected with other agents' internal agents. Figure 5 represents the hierarchical structure of SMACS framework, where all internal agents are grouped together from Monitor Int-Agent to Executer Int-Agent.

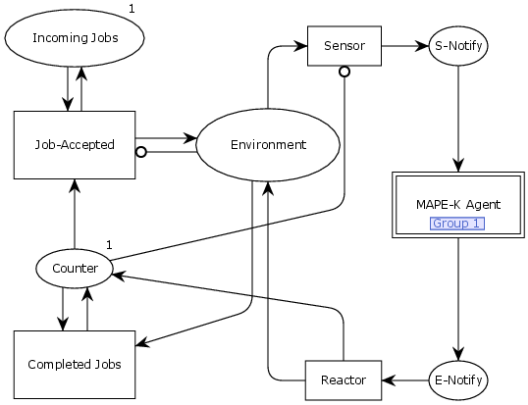



**FIGURE 5.** Hierarchical architecture of Figure 4.

### B. SPECIFICATION OF SMACS FRAMEWORK

CPN does not provide the flexibility to model self-adaptive behavior of any structure. To overcome this difficulty we proposed modal $\mu$-calculus, which provides a range of valuable recursive properties. I$\mu$TL is more expressive than $\mu$-calculus but unavailability of its model checker we are using $\mathcal{M}_\mu$ for the specification of all internal agents of SMACS framework. The result shows that how an internal agent interact with the environment and also interact with other agent intelligently. All these properties will be verified through the TAPA model checker. Figure 6 represents the working of environment.

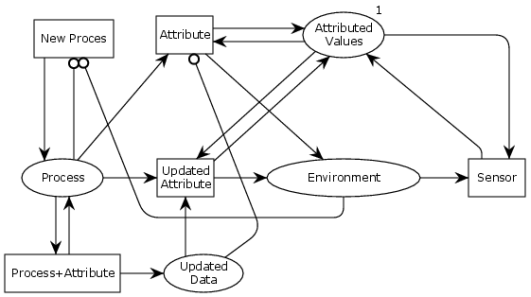



**FIGURE 6.** PN model for environment.

### 1) SPECIFICATION FOR ENVIRONMENT

Figure 6 represents Environment architecture and specification in $\mathcal{M}_\mu$ is given below:

$$[\![\langle NP?\rangle true \wedge (\neg\langle UpAtt!\rangle false \wedge \langle Att!\rangle true)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\langle NP?\rangle true \wedge ([UpAtt!](\langle P?\rangle true \wedge \langle Att?\rangle true))]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_\mu.(\langle UpAtt\rangle\mathcal{Z}_\mu \wedge (\langle Att\rangle\mathcal{Z}_\mu \vee \langle P\rangle\mathcal{Z}_\mu)) \vee \nu\mathcal{Y}_\mu$$
$$.([-tau]false \wedge (\langle tau\rangle true \wedge [tau]\mathcal{Y}_\mu))]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_\mu.([P]false \vee [Att]false \vee [UpAtt]false) \vee \langle *\rangle\mathcal{Z}_\mu]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_\mu.([UpAtt](\langle P\rangle true \wedge \langle Att\rangle true) \wedge [*]\mathcal{Z}_\mu]\!]^{\delta}_{\mathcal{N}}$$

### 2) SPECIFICATION FOR MONITOR

Figure 7 represents Monitor Int-Agent model and specification in $\mathcal{M}_\mu$ is given below:

$$[\![[S?](\langle SS?\rangle true \wedge \neg\langle CT?\rangle false) \wedge \neg\langle FR!\rangle false]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[SS?](\langle CT?\rangle true \wedge \neg\langle S?\rangle false \wedge \neg\langle FR!\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[CT?](\langle FR!\rangle true \wedge \neg\langle S?\rangle false \wedge \neg\langle SS?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[FR!](\langle S?\rangle true \wedge \neg\langle CT?\rangle false \wedge \neg\langle SS?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_\mu.(\langle FR\rangle\mathcal{Z}_\mu \wedge (\langle SS\rangle\mathcal{Z}_\mu \vee \langle CT\rangle\mathcal{Z}_\mu)) \vee \nu\mathcal{Y}_\mu$$
$$.([-tau]false) \wedge (\langle tau\rangle true) \wedge ([tau]\mathcal{Y}_\mu)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_\mu.([SS]false \vee [CT]false \vee [FR]false) \vee \langle *\rangle\mathcal{Z}_\mu]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\nu\mathcal{Z}_\mu.([FR](\langle SS\rangle true \wedge \langle CT\rangle true) \wedge [*]\mathcal{Z}_\mu]\!]^{\delta}_{\mathcal{N}}$$

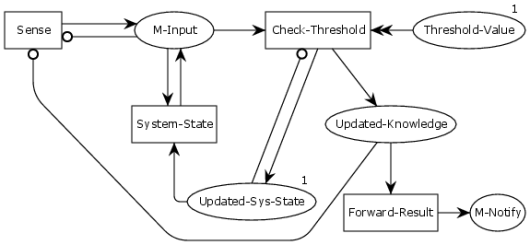



**FIGURE 7.** PN model for monitor.

### 3) SPECIFICATION FOR ANALYZER

Figure 8 represents Analyzer Int-Agent model and specification in $\mathcal{M}_\mu$ is given below:

$$[\![[SDR?](\langle SSym?\rangle true \wedge \neg\langle USR?\rangle false$$
$$\wedge\neg\langle UDR!\rangle false\neg\langle FReq!\rangle false)]\!]^{\delta}_{\mathcal{N}}$$

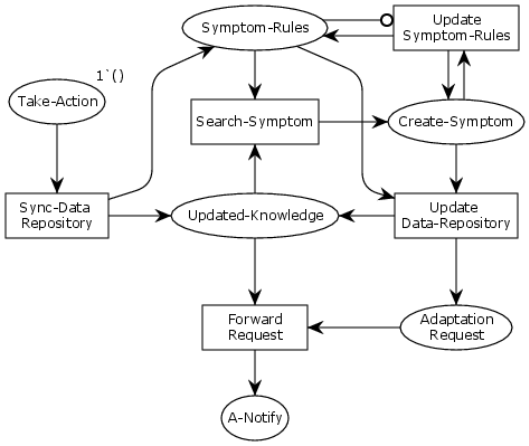



**FIGURE 8.** PN model for analyzer.

$$[\![[SSym?](\langle USR?\rangle true \wedge \neg\langle UDR!\rangle false$$
$$\wedge\neg\langle SDR?\rangle false\neg\langle FReq!\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[USR?](\langle UDR!\rangle true \wedge \neg\langle USR?\rangle false$$
$$\wedge\neg\langle SDR?\rangle false\neg\langle FReq!\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[UDR!](\langle FReq!\rangle true \wedge \neg\langle SSym?\rangle false$$
$$\wedge\neg\langle SDR?\rangle false\neg\langle USR?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[FReq!](\langle SDR?\rangle true \wedge \neg\langle USR?\rangle false \wedge \neg\langle UDR!\rangle false$$
$$\neg\langle SSym?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_\mu.(\langle FReq\rangle\mathcal{Z}_\mu \wedge (\langle USR\rangle\mathcal{Z}_\mu \vee \langle SDR\rangle\mathcal{Z}_\mu \vee \langle UDR\rangle$$
$$\mathcal{Z}_\mu \vee \langle SSym\rangle\mathcal{Z}_\mu)) \vee \nu\mathcal{Y}_\mu.([-tau]false) \wedge (\langle tau\rangle true)$$
$$\wedge([tau]\mathcal{Y}_\mu)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_\mu.([SDR]false \vee [SSym]false \vee [USR]false$$
$$\vee[UDR]false \vee [FReq]false) \vee \langle *\rangle\mathcal{Z}_\mu]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\nu\mathcal{Z}_\mu.([FReq](\langle SDR\rangle true \wedge \langle SSym\rangle true \wedge \langle USR\rangle true$$
$$\wedge\langle UDR\rangle) \wedge [*]\mathcal{Z}_\mu]\!]^{\delta}_{\mathcal{N}}$$

### 4) SPECIFICATION FOR PLANNER

Figure 9 represents Planner Int-Agent model and specification in $\mathcal{M}_\mu$ is given below:

$$[\![[MR?](\langle AR?\rangle true \wedge \neg\langle RR?\rangle false) \wedge \neg\langle Add!\rangle false]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[AR?](\langle RR?\rangle true \wedge \neg\langle MR?\rangle false \wedge \neg\langle Add!\rangle false)]\!]^{\delta}_{\mathcal{N}}$$

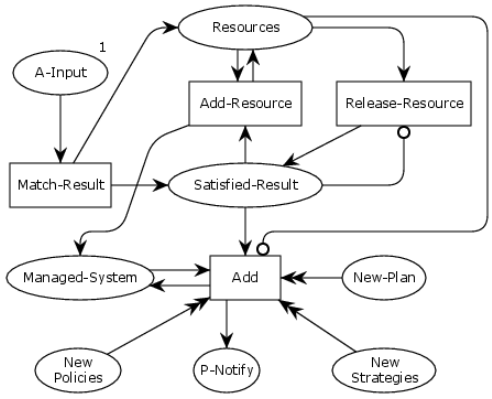



**FIGURE 9.** PN model for planner.

$$[\![[RR?](\langle Add!\rangle true \wedge \neg\langle MR?\rangle false \wedge \neg\langle AR?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[Add!](\langle MR?\rangle true \wedge \neg\langle AR?\rangle false \wedge \neg\langle RR?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_{\mu}.(\langle Add\rangle\mathcal{Z}_{\mu} \wedge (\langle MR\rangle\mathcal{Z}_{\mu} \vee \langle AR\rangle\mathcal{Z}_{\mu} \vee \langle RR\rangle\mathcal{Z}_{\mu})) \vee \nu\mathcal{Y}_{\mu}.([-tau]false) \wedge (\langle tau\rangle true) \wedge ([tau]\mathcal{Y}_{\mu})]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_{\mu}.([Add]false \vee [MR]false \vee [AR]false \vee [RR]flase) \vee \langle *\rangle\mathcal{Z}_{\mu}]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\nu\mathcal{Z}_{\mu}.([Add](\langle MR\rangle true \wedge \langle AR\rangle true \wedge \langle RR\rangle true) \wedge [*]\mathcal{Z}_{\mu}]\!]^{\delta}_{\mathcal{N}}$$

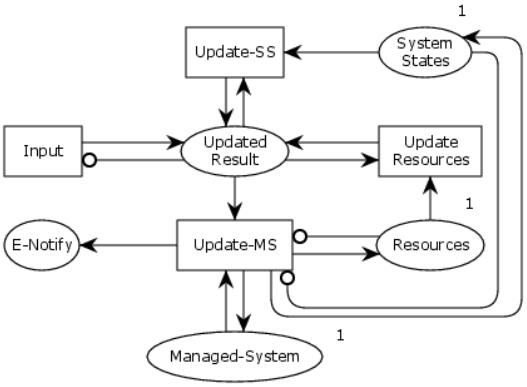



**FIGURE 10.** PN model for executer.

### 5) SPECIFICATION FOR EXECUTER

Figure 10 represents Executer Int-Agent model and specification in $\mathcal{M}_{\mu}$ is given below:

$$[\![[Input?](\langle USS?\rangle true \wedge \langle UR?\rangle true) \wedge \neg\langle UMS!\rangle false]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[USS?](\langle UR?\rangle true \wedge \neg\langle UMS!\rangle false \wedge \neg\langle Input?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[UR?](\langle USS?\rangle true \wedge \neg\langle UMS!\rangle false \wedge \neg\langle Input?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![[UMS!](\langle Input?\rangle true \wedge \neg\langle USS?\rangle false \wedge \neg\langle UR?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_{\mu}.(\langle UMS\rangle\mathcal{Z}_{\mu} \wedge (\langle Input\rangle\mathcal{Z}_{\mu} \vee \langle USS\rangle\mathcal{Z}_{\mu} \vee \langle UR\rangle\mathcal{Z}_{\mu})) \vee \nu\mathcal{Y}_{\mu}.([-tau]false) \wedge (\langle tau\rangle true) \wedge ([tau]\mathcal{Y}_{\mu})]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\mu\mathcal{Z}_{\mu}.([UMS]false \vee [Input]false \vee [USS]false \vee [UR]flase) \vee \langle *\rangle\mathcal{Z}_{\mu}]\!]^{\delta}_{\mathcal{N}}$$
$$[\![\nu\mathcal{Z}_{\mu}.([UMS](\langle Input\rangle true \wedge \langle USS\rangle true \wedge \langle UR\rangle true) \wedge [*]\mathcal{Z}_{\mu}]\!]^{\delta}_{\mathcal{N}}$$

The Table 1 represents all the abbreviations used in $\mathcal{M}_{\mu}$ formulae discussed in section III.

## IV. VERIFICATION OF SMACS FRAMEWORK

In the proposed research, we are using TAPA model checker to verify the properties of each internal agent of SMACS framework. The properties are syntactically modified according to the model checker for their execution. This syntactic change will not affect the semantic of internal agents. For verification of internal agents' properties we are taking Traffic Monitoring System (TMS) as a case study. The proposed monitoring system is specifically for Lahore city. Due to a huge rush of traffic almost in every area, this city needs an effective traffic monitoring system to overcome such kind of problem. It is also painful when someone losses his/her life as an emergency vehicle cannot reach to its destination due to huge traffic jam. The proposed traffic management system in this research will also cover such problems specifically.

**TABLE 1.** Abbreviation used in $\mathcal{M}_{\mu}$ formulae.

| Name | Description |
|---|---|
| NP | New Process |
| Att | Attribute |
| UPAtt | Updated Attribute |
| P | Process |
| S | Sense |
| SS | System-State |
| CT | Check-Threshold |
| FR | Forward Result |
| SDR | Sync-Data Repository |
| SSym | Search-Symptom |
| USR | Updated Symptom-Rules |
| UDR | Updated Data-Repository |
| FReq | Forward Request |
| MR | Match-Result |
| AR | Add-Resource |
| RR | Release-Resource |
| USS | Update System States |
| UR | Update Resources |
| UMS | Update Managed System |

We are choosing Faisal Chowk signal at Shahrah-e-Quaid-i-Azam for implementing our SMACS framework. At this point two roads are connected with Shahrah-e-Quaid-i-Azam (MR1, MR2), which are Edgerton road (ER) and Queen's road (QR) as shown in Figure 11. According to this there can

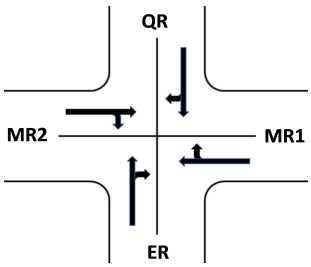



**FIGURE 11.** Traffic signal structure for Faisal Chowk at Shahrah-e-Quaid-i-Azam.

**FIGURE 12.** Concurrent behavior of traffic signals.

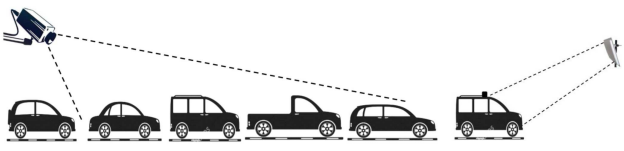


**FIGURE 13.** Camera view for detecting vehicles (left) and E-tag detector (right).

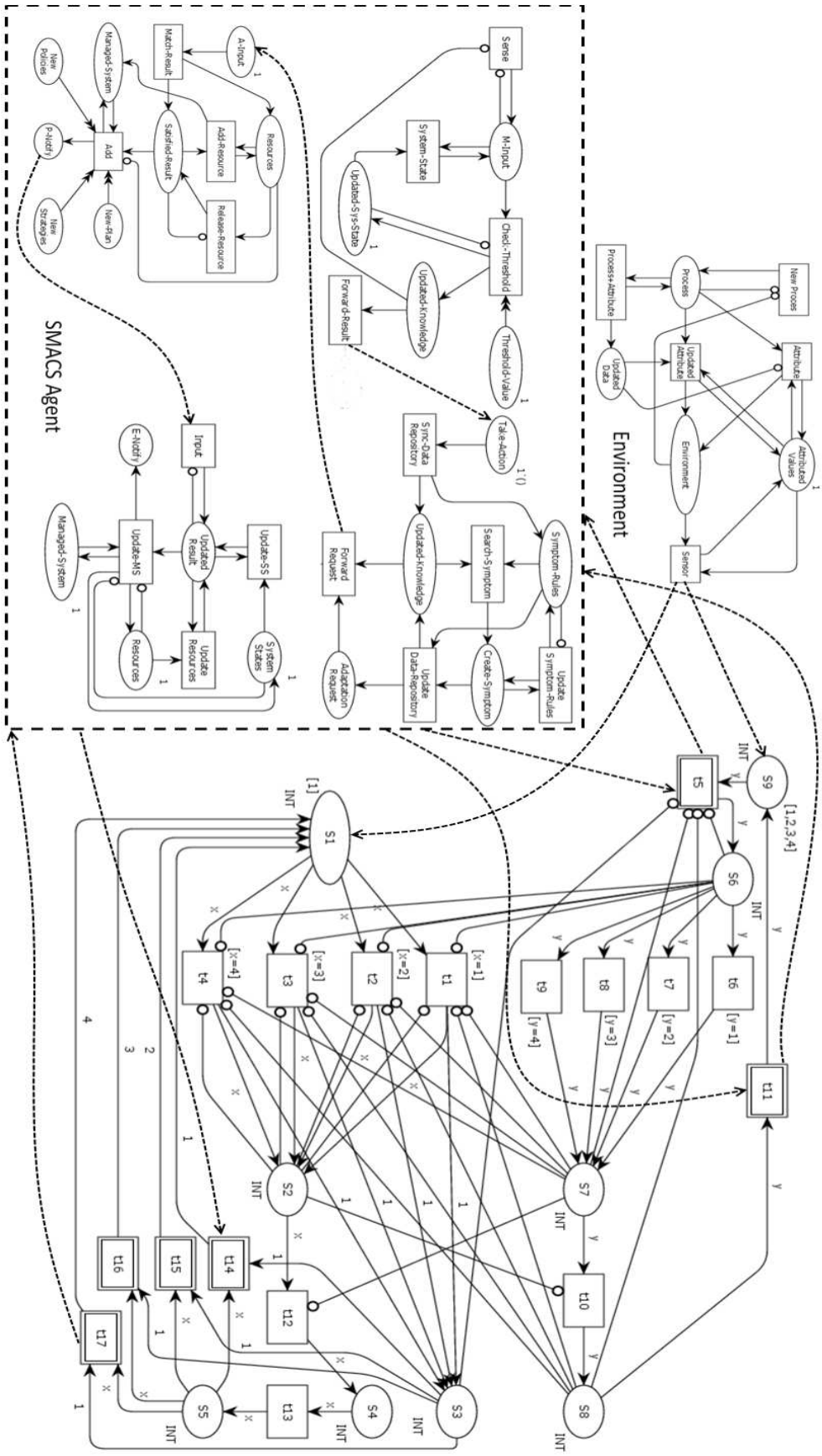



**FIGURE 14.** A CPN model for traffic monitoring system using SMACS framework.

**TABLE 2.** Modified $\mathcal{M}_\mu$ formulae for TAPA model checker when emergency vehicle detected at ER side.

| Property Name | Modified Formula for TAPA | Description |
|---|---|---|
| En1 | $\langle NP?\rangle true\&(!\langle UpAtt!\rangle false\&\langle Att!\rangle true)$ | New process doesn't need updated value |
| En2 | $\langle NP?\rangle true\&([UpAtt!](\langle P?\rangle true\&\langle Att?\rangle true))$ | New process needs updated value |
| Input forwarded to S1 and S9 places through Sensor channel. The specification below shows the working of TMS when emergency vehicle detected at place S9. | | |
| TMS1 | $[Em\_I\_ER?]true\&(!\langle N\_I\_QR?\rangle false\& !\langle N\_I\_MR1?\rangle false\&!\langle N\_I\_MR2?\rangle false)$ | Emergency vehicle is detected and signal will become green after getting updates from MAPE-K internal agents |
| M1 | $[S?](\langle SS?\rangle true\&!\langle CT?\rangle false\&!\langle FR!\rangle false)$ | Monitor receives job through S9 place |
| M2 | $[SS?](\langle CT?\rangle true\&!\langle S?\rangle false\&!\langle FR!\rangle false)$ | Updated system states are checked |
| M3 | $[CT?](\langle FR!\rangle true\&!\langle S?\rangle false\&!\langle SS?\rangle false)$ | Threshold values are checked |
| M4 | $[FR!](\langle S?\rangle true\&!\langle CT?\rangle false\&!\langle SS?\rangle false)$ | Result is Forwarded to Analyzer Int-Agent |
| A1 | $[SDR?](\langle SSym?\rangle true\&!\langle USR?\rangle false\& !\langle UDR!\rangle false!\langle FReq!\rangle false)$ | Synchronize input with data repository |
| A2 | $[SSym?](\langle USR?\rangle true\&!\langle UDR!\rangle false\& !\langle SDR?\rangle false!\langle FReq!\rangle false)$ | Symptoms are searched for input |
| A3 | $[USR?](\langle UDR!\rangle true\&!\langle USR?\rangle false\& !\langle SDR?\rangle false!\langle FReq!\rangle false)$ | Update symptom rules if needed |
| A4 | $[UDR!](\langle FReq!\rangle true\&!\langle SSym?\rangle false\& !\langle SDR?\rangle false!\langle USR?\rangle false)$ | Update data repository if new symptom rules are created |
| A5 | $[FReq!](\langle SDR?\rangle true\&!\langle USR?\rangle false\& !\langle UDR!\rangle false!\langle SSym?\rangle false)$ | Forwarded request to Plan Int-Agent |
| P1 | $[MR?](\langle AR?\rangle true\&!\langle RR?\rangle false\&!\langle Add!\rangle false)$ | Results received from previous internal a-gent are matched |
| P2 | $[AR?](\langle RR?\rangle true\&!\langle MR?\rangle false\&!\langle Add!\rangle false)$ | Add new resources if needed |
| P3 | $[RR?](\langle Add!\rangle true\&!\langle MR?\rangle false\&!\langle AR?\rangle false)$ | Release resources if added |
| P4 | $[[Add!](\langle MR?\rangle true\&!\langle AR?\rangle false\&!\langle RR?\rangle false)$ | Add new plans, new strategies and new policies if needed |
| Ex1 | $[Input?](\langle USS?\rangle true\&\langle UR?\rangle true)\& !\langle UMS!\rangle false)$ | Accept input from Plan Int-Agent |
| Ex2 | $[USS?](\langle UR?\rangle true\&!\langle UMS!\rangle false\& !\langle Input?\rangle false)$ | System states are updated |
| Ex3 | $[UR?](\langle USS?\rangle true\&!\langle UMS!\rangle false\& !\langle Input?\rangle false)$ | Resources are updated |
| Ex4 | $[UMS!](\langle Input?\rangle true\&!\langle USS?\rangle false\& !\langle UR?\rangle false)$ | Managed system is updated and result is forwarded, so Signal would be yellow at place S6 green at place S7 on ER side |
| TMS2 | $[Em\_O\_ER!]true\&(\langle N\_I\_QR?\rangle true \|\langle N\_I\_MR1?\rangle true\|\langle N\_I\_MR2?\rangle true)$ | All other signal will become to normal status when emergency vehicle will cross the signal |

be four type of signal modeling; at MR1 vehicles can move straight as well as to right turn on green light, similarly at MR2, ER and QR can be repeated same criteria. There can be applied two types of sensor agents to monitor signals; video cameras and e-tags (Radio Frequency Identification "RFID" chips) which are specific for ambulances and all other emergency vehicles. Light sensor can also be applied as sensor agents for detecting blue or red revolving light mostly used by emergency vehicles. Concurrently three types of signal modeling can be applied after watching the traffic load by sensing devices as shown in Figure 12.

In case of emergency, the emergency vehicle will be detected either by e-tag antenna or by light detector, which are being worked as self-adaptive agents. The signal will

**TABLE 3.** Modified $\mathcal{M}_\mu$ formulae for TAPA model checker when congestion occurs at ER side.

| Property Name | Modified Formula for TAPA | Description |
|---|---|---|
| En1 | $\langle NP?\rangle true\&(!\langle UpAtt!\rangle false\&\langle Att!\rangle true)$ | New process doesn't need updated value |
| En2 | $\langle NP?\rangle true\&([UpAtt!](\langle P?\rangle true\&\langle Att?\rangle true))$ | New process needs updated value |
| At place S5 when congestion will be detected at ER side, SMACS agent will be activated at transition t14. The specification below shows the working of TMS at ER side during congestion. | | |
| TMS1 | $[Con\_I\_ER?]\&(!\langle N\_I\_QR?\rangle false\&$ $!\langle N\_I\_MR1?\rangle false\&!\langle N\_I\_MR2?\rangle false)$ | Congestion on one side preferably opens signal |
| M1 | $[S?](\langle SS?\rangle true\&!\langle CT?\rangle false\&!\langle FR!\rangle false)$ | Monitor receives job through S9 place |
| M2 | $[SS?](\langle CT?\rangle true\&!\langle S?\rangle false\&!\langle FR!\rangle false)$ | Updated system states are checked |
| M3 | $[CT?](\langle FR!\rangle true\&!\langle S?\rangle false\&!\langle SS?\rangle false)$ | Threshold values are checked |
| M4 | $[FR!](\langle S?\rangle true\&!\langle CT?\rangle false\&!\langle SS?\rangle false)$ | Result is Forwarded to Analyzer Int-Agent |
| A1 | $[SDR?](\langle SSym?\rangle true\&!\langle USR?\rangle false\&$ $!\langle UDR!\rangle false!\langle FReq!\rangle false)$ | Synchronize input with data repository |
| A2 | $[SSym?](\langle USR?\rangle true\&!\langle UDR!\rangle false\&$ $!\langle SDR?\rangle false!\langle FReq!\rangle false)$ | Symptoms are searched for input |
| A3 | $[USR?](\langle UDR!\rangle true\&!\langle USR?\rangle false\&$ $!\langle SDR?\rangle false!\langle FReq!\rangle false)$ | Update symptom rules if needed |
| A4 | $[UDR!](\langle FReq!\rangle true\&!\langle SSym?\rangle false\&$ $!\langle SDR?\rangle false!\langle USR?\rangle false)$ | Update data repository if new symptom rules are created |
| A5 | $[FReq!](\langle SDR?\rangle true\&!\langle USR?\rangle false\&$ $!\langle UDR!\rangle false!\langle SSym?\rangle false)$ | Forwarded request to Plan Int-Agent |
| P1 | $[MR?](\langle AR?\rangle true\&!\langle RR?\rangle false\&!\langle Add!\rangle false)$ | Results received from previous internal agent are matched |
| P2 | $[AR?](\langle RR?\rangle true\&!\langle MR?\rangle false\&!\langle Add!\rangle false)$ | Add new resources if needed |
| P3 | $[RR?](\langle Add!\rangle true\&!\langle MR?\rangle false\&!\langle AR?\rangle false)$ | Release resources if added |
| P4 | $[[Add!](\langle MR?\rangle true\&!\langle AR?\rangle false\&!\langle RR?\rangle false)$ | Add new plans, new strategies and new policies if needed |
| Ex1 | $[Input?](\langle USS?\rangle true\&\langle UR?\rangle true)\&$ $!\langle UMS!\rangle false)$ | Accept input from Plan Int-Agent |
| Ex2 | $[USS?](\langle UR?\rangle true\&!\langle UMS!\rangle false\&$ $!\langle Input?\rangle false)$ | System states are updated |
| Ex3 | $[UR?](\langle USS?\rangle true\&!\langle UMS!\rangle false\&$ $!\langle Input?\rangle false)$ | Resources are updated |
| Ex4 | $[UMS!](\langle Input?\rangle true\&!\langle USS?\rangle false\&$ $!\langle UR?\rangle false)$ | Managed system is updated and result is forwarded, so Signal would be yellow at place S6 green at place S7 on ER side |
| TMS2 | $[Con\_O\_ER!]true\&(\langle N\_I\_QR?\rangle true$ $\|\langle N\_I\_MR1?\rangle true\|\langle N\_I\_MR2?\rangle true)$ | All other signal will become to normal status when congestion problem will be solved |

become green and an update is forwarded to all other sensing agents. When the emergency vehicle will cross the signal another update will be forwarded to all sensing agents by the agent monitoring that vehicle and routine jobs will be started. In case of high traffic load or an emergency vehicle detected on any side as represented in Figure 13, the signal will be preferably opened by receiving updates through video cameras or RFID detectors.

A CPN based model is represented in Figure 14 for TMS using SMACS framework. There are about 6 self-adaptive agents working for monitoring and managing traffic at Faisal Chowk signal. Each SMACS agent will update to other agents after performing any task. According to figure t1, t2, t3 and t4 transitions are being used for controlling signals, t14, t15, t16 and t17 agents are being used to monitor the capacity of vehicles with the help of video camera based sensing devices

**TABLE 4.** Modified $\mathcal{M}_\mu$ formulae for TAPA model checker to check liveness, safeness and deadlock-freedom properties of TMS.

| Property Name | Modified Formula for TAPA | Description |
|---|---|---|
| TMS4 | $!minZ.(\langle Em\_I\_ER\rangle Z\&(\langle Con\_I\_ER\rangle Z \mid\langle N\_I\_ER\rangle Z\mid\langle Em\_O\_ER\rangle Z\mid\langle Con\_O\_ER\rangle Z \mid\langle N\_O\_ER\rangle Z))\mid maxY.([-tau]false)\&(\langle tau\rangle true)\&([tau]Y)$ | Liveness property is checked for TMS at ER side |
| TMS5 | $minZ.([Em\_I\_ER]false\mid[Con\_I\_ER]false \mid[N\_I\_ER]false)\mid[Em\_O\_ER]false) \mid[Con\_O\_ER]false)\mid[N\_O\_ER]false)\mid\langle *\rangle Z$ | Safety property is checked for TMS at ER side |
| TMS6 | $maxZ.([Em\_I\_ER](\langle Con\_I\_ER\rangle true\& \langle N\_I\_ER\rangle true\&\langle Em\_O\_ER\rangle true \&\langle Con\_O\_ER\rangle true\&\langle N\_O\_ER\rangle true)\&[*]Z)$ | Deadlock freedom property is checked for TMS at ER side |
| **Liveness**, **Safeness** and **Deadlock-freedom** properties for SMACS's internal agents. | | |
| M1 | $!minZ.(\langle FR\rangle Z\&(\langle SS\rangle Z\mid\langle CT\rangle Z))\mid maxY.([-tau]false)\&(\langle tau\rangle true)\&([tau]Y)$ | Liveness property is checked for Monitor |
| M2 | $minZ.([SS]false\mid[CT]false\mid[FR]false)\mid\langle *\rangle Z$ | Safety property is checked for Monitor |
| M3 | $maxZ.([FR](\langle SS\rangle true\&\langle CT\rangle true)\&[*]Z)$ | Deadlock freedom property is checked for Monitor |
| A1 | $!minZ.(\langle FReq\rangle Z\&(\langle USR\rangle Z\mid\langle SDR\rangle Z\mid\langle UDR\rangle Z \mid\langle SSym\rangle Z))\mid maxY.([-tau]false) \&(\langle tau\rangle true)\&([tau]Y)$ | Liveness property is checked for Analyzer |
| A2 | $minZ.([SDR]false\mid[SSym]false\mid[USR]false \mid[UDR]false\mid[FReq]false)\mid\langle *\rangle Z$ | Safety property is checked for Analyzer |
| A3 | $maxZ.[FReq](\langle SDR\rangle true\&\langle SSym\rangle true\& \langle USR\rangle true\&\langle UDR\rangle)\&[*]Z$ | Deadlock freedom property is checked for Analyzer |
| P1 | $!minZ.(\langle Add\rangle Z\&(\langle MR\rangle Z\mid\langle AR\rangle Z\mid\langle RR\rangle Z)) \mid maxY.([-tau]false)\&(\langle tau\rangle true)\&([tau]Y)$ | Liveness property is checked for Planner |
| P2 | $minZ.([Add]false\mid[MR]false\mid[AR]false \mid[RR]flase)\mid\langle *\rangle Z$ | Safety property is checked for Planner |
| P3 | $maxZ.[Add](\langle MR\rangle true\&\langle AR\rangle true\&\langle RR\rangle true) \&[*]Z$ | Deadlock freedom property is checked for Planner |
| Ex1 | $!minZ.(\langle UMS\rangle Z\&(\langle Input\rangle Z\mid\langle USS\rangle Z\mid\langle UR\rangle Z)) maxY.([-tau]false)\&(\langle tau\rangle true)\&([tau]Y)$ | Liveness property is checked for Executer |
| Ex2 | $minZ.([UMS]false\mid[Input]false\mid[USS]false\mid [UR]flase)\mid\langle *\rangle Z$ | Safety property is checked for Executer |
| Ex3 | $maxZ.([UMS]\langle Input\rangle true\&\langle USS\rangle true\& \langle UR\rangle true)\&[*]Z$ | Deadlock freedom property is checked for Executer |

on ER, MR1, QR and MR2 respectively. Similarly t5 agent is being used specifically for controlling emergency vehicles, as any emergency vehicle detected through e-tag, the control of all signals is transferred to t5 self-adaptively. Updates of signal status is forwarded to S7 through firing one of the transition (t6, t7, t8 or t9). After firing t10 the signal status at place S8 represents an open path for emergency vehicle and the control is shifted back to normal working after getting updates from t11 agents. For modeling of internal agents' structure $\mathcal{M}_\mu$ is used and for verification, we are using TAPA Model checker. The Table 2 and Table 3 represent the converted semantic of $\mathcal{M}_\mu$ written according to the TAPA model checker when an emergency vehicle is detected and when congestion is occurred at ER side respectively. Similarly, Table 4 represent the converted semantic of $\mathcal{M}_\mu$ written according to TAPA model checker for checking liveness, safeness and deadlock-freedom properties of TMS at ER side and all internal agents of SMACS agent. S1 and S9 are actually the environment places, whenever a new process generated the sensor channel detect and inform to SMACS agent. S1 place contains signal initial status, which is *Red*, after activating signal control agents, the status of agent is converted to *Yellow* at place S2 and after some delay at t12 the signal is shifted to *Green* at place S4. The Green signal status will remain for about 20 to 60 seconds or according to traffic load. This long delay is calculated at t1, and then status of signal is shifted to yellow at place S5. After this SMACS agents decide to run next signal turn by turn or to run for specific side where traffic load is more than normal condition.

Atomic propositions for TMS are {Normal-Input, Emergency-Input, Congestion-Input, Normal-Exit, Emergency-Exit, Congestion-Exit}, and signals condition

should be {Red, Em-Yellow, Yellow, Em-Green, Green, Yellow-Out}. The following specification of TMS represents that signal on ER side for emergency vehicles or congestion side will be opened and all other sides should be closed until the emergency vehicle cross the signal and congestion will be decreased to normal condition.

$$[\![[Em\_I\_ER?]true \wedge (\neg\langle N\_I\_QR?\rangle false \wedge \neg\langle N\_I\_MR1?\rangle false \wedge \neg\langle N\_I\_MR2?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$

$$[\![[Em\_O\_ER!]true \wedge (\langle N\_I\_QR?\rangle true \vee \langle N\_I\_MR1?\rangle true \vee \langle N\_I\_MR2?\rangle true)]\!]^{\delta}_{\mathcal{N}}$$

$$[\![[Con\_I\_ER?]true \wedge (\neg\langle N\_I\_QR?\rangle false \wedge \neg\langle N\_I\_MR1?\rangle false \wedge \neg\langle N\_I\_MR2?\rangle false)]\!]^{\delta}_{\mathcal{N}}$$

$$[\![[Con\_O\_ER!]true \wedge (\langle N\_I\_QR?\rangle true \vee \langle N\_I\_MR1?\rangle true \vee \langle N\_I\_MR2?\rangle true)]\!]^{\delta}_{\mathcal{N}}$$

First property shows that after detecting an emergency vehicle by e-tag sensor agent on ER side, then all other sides vehicles must be blocked and second property shows that after crossing emergency vehicle, any other signal (QR, MR1 or MR2) will be turned to green. Similarly, third and fourth properties show that during congestion on ER side, the signal of that side will be preferably opened and remain open until traffic load will become to normal condition. During critical situations signals will be managed concurrently according to Figure 12 and in normal situations signals will be worked smoothly, i.e. first MR1 will become green, then ER, then MR2 and then QR. Through this case study it is verified that the soundness and the completeness of the SMACS framework is fully addressed.

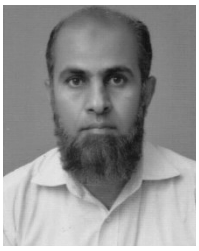
## V. CONCLUSION

In the proposed Self-adaptive Multi-Agent Concurrent System (SMACS) framework the complex concurrent behavior has been studied, whereas MAPE-K feedback loop is employed for modeling of internal structure of self-adaptive agents, that possess complex concurrent behavior. In SMACS framework any change is detected in the environment through the sensor channel, while the corresponding output is interacted with the environment through the reactor channel. The SMACS framework have four kinds of internal agents, namely: Monitor Int-Agent, Analyzer Int-Agent, Planner Int-Agent and Executer Int-Agent. These agents interacted with each other through managed system and knowledge based updates. Also, there are two more channels, the emitter and the consenter are used for inter-communication between SMACS agents. In order to express the internal behavior of each agent, the Modal $\mu$-calculus while for the verification of their internal properties TAPA model checker are used. The traffic monitoring and management system have been selected for the implementation of the SMACS framework. This model also addressed the handling of emergency vehicles during congestion on a signal. In order to monitor signals two kinds of sensing devices are used, which are video cameras and e-tags antennas. The e-tag antennas are specifically used for ambulances and all other emergency vehicles. The light sensor also applied as a sensing device for detecting blue or red revolving light mostly used by emergency vehicles. It is formally verified that the proposed work is suitable to prove correctness of the system having complex concurrent behavior. Moreover, the liveness, the safety and the deadlock-freedom properties are also verified. In our future work we will try to apply this framework for communication between multiple self-adaptive systems using diverse agent platforms.

## REFERENCES

[1] P. A. Abdulla, B. Jonsson, M. Nilsson, and M. Saksena, "A survey of regular model checking," in *Proc. Int. Conf. Concurrency Theory*. Berlin, Germany: Springer, 2004, pp. 35–48.
[2] H. R. Andersen, "Model checking and Boolean graphs," *Theor. Comput. Sci.*, vol. 126, no. 1, pp. 3–30, 1994.
[3] E. M. Clarke and E. A. Emerson, "Design and synthesis of synchronization skeletons using branching time temporal logic," in *25 Years Model Checking*. Berlin, Germany: Springer, 1981, pp. 52–71.
[4] T. Schuele and K. Schneider, "Bounded model checking of infinite state systems," *Formal Methods Syst. Des.*, vol. 30, no. 1, pp. 51–81, 2007.
[5] M. A. Niazi, "Introduction to the modeling and analysis of complex systems: A review," *Complex Adapt. Syst. Model.*, Feb. 2016.
[6] M. A. Niazi and A. Hussain, "Complex adaptive systems," in *Cognitive Agent-Based Computing-I*. Dordrecht, The Netherlands: Springer, 2013, pp. 21–32.
[7] K. Jensen, L. M. Kristensen, and L. Wells, "Coloured Petri nets and CPN tools for modelling and validation of concurrent systems," *Int. J. Softw. Tools Technol. Transf.*, vol. 9, nos. 3–4, pp. 213–254, 2007.
[8] F. Calzolai, R. De Nicola, M. Loreti, and F. Tiezzi, "TAPAs: A tool for the analysis of process algebras," in *Transactions on Petri Nets and Other Models of Concurrency*. Berlin, Germany: Springer, 2008, pp. 54–70.
[9] D. Kozen, "Results on the propositional $\mu$-calculus," *Theor. Comput. Sci.*, vol. 27, no. 3, pp. 333–354, 1983.
[10] S. A. R. Kazmi and W. H. Zhang, "Compositional reasoning in intuitionistic linear time mu-calculus," (in Chinese), *J. Softw.*, vol. 20, no. 8, pp. 2026–2036, 2009. [Online]. Available: http://www.jos.org.cn/josen/ch/reader/key_query.aspx#
[11] I. Fakhir, S. A. R. Kazmi, A. Qasim, and I. Rafique, "Concurrency in intuitionistic linear-time $\mu$-calculus: A case study of manufacturing system," *Indian J. Sci. Technol.*, vol. 9, no. 6, pp. 1–7, 2016.
[12] V. Botti, C. Carrascosa, V. Julián, and J. Soler, "Modelling agents in hard real-time environments," in *Proc. Eur. Workshop Modeling Auto. Agents Multi-Agent World*. Berlin, Germany: Springer, 1999, pp. 63–76.
[13] Y. Brun *et al.*, "Engineering self-adaptive systems through feedback loops," in *Software Engineering for Self-Adaptive Systems*. Berlin, Germany: Springer, 2009, pp. 48–70.
[14] P. Arcaini, E. Riccobene, and P. Scandurra, "Modeling and analyzing MAPE-K feedback loops for self-adaptation," in *Proc. 10th Int. Symp. Softw. Eng. Adapt. Self-Manage. Syst.*, May 2015, pp. 13–23.
[15] D. G. D. L. Iglesia and D. Weyns, "MAPE-K formal templates to rigorously design behaviors for self-adaptive systems," *ACM Trans. Auton. Adapt. Syst.*, vol. 10, no. 3, 2015, Art. no. 15.
[16] M. U. Iftikhar and D. Weyns. (2012). "A case study on formal verification of self-adaptive behaviors in a decentralized system." [Online]. Available: https://arxiv.org/abs/1208.4635
[17] D. Weyns, S. Malek, and J. Andersson, "FORMS: A formal reference model for self-adaptation," in *Proc. 7th Int. Conf. Auton. Comput.*, 2010, pp. 205–214.
[18] P. Vromant, D. Weyns, S. Malek, and J. Andersson, "On interacting control loops in self-adaptive systems," in *Proc. 6th Int. Symp. Softw. Eng. Adapt. Self-Manage. Syst.*, 2011, pp. 202–207.
[19] M. U. Iftikhar and D. Weyns, "ActivFORMS: Active formal models for self-adaptation," in *Proc. 9th Int. Symp. Softw. Eng. Adapt. Self-Manage. Syst.*, 2014, pp. 125–134.
[20] P. Arcaini, E. Riccobene, and P. Scandurra, "Formal design and verification of self-adaptive systems with decentralized control," *ACM Trans. Auto. Adapt. Syst.*, vol. 11, no. 4, 2017, Art. no. 25.
[21] M. A. Niazi, A. Hussain, and M. Kolberg. (2017). "Verification & validation of agent based simulations using the VOMAS (virtual overlay multi-agent system) approach." [Online]. Available: https://arxiv.org/abs/1708.02361

[22] R. Adler, I. Schaefer, T. Schuele, and E. Vecchié, "From model-based design to formal verification of adaptive embedded systems," in *Proc. Int. Conf. Formal Eng. Methods*. Berlin, Germany: Springer, 2007, pp. 76–95.
[23] E. Merelli, N. Paoletti, and L. Tesei. (2012). "A multi-level model for self-adaptive systems." [Online]. Available: https://arxiv.org/abs/1209.1628
[24] S. Loukil, S. Kallel, and M. Jmaiel, "An approach based on runtime models for developing dynamically adaptive systems," *Future Gener. Comput. Syst.*, vol. 68, pp. 365–375, Mar. 2017.
[25] G. Smith and K. Winter, "Incremental development of multi-agent systems in object-Z," in *Proc. 35th Annu. Softw. Eng. Workshop*, Oct. 2012, pp. 120–129.
[26] T. Miller and P. McBurney, "Multi-agent system specification using TCOZ," (in German), in *Proc. Conf. Multiagent Syst. Technol.* Berlin, Germany: Springer, 2005, pp. 216–221.
[27] A. Qasim and S. A. R. Kazmi, "MAPE-K interfaces for formal modeling of real-time self-adaptive multi-agent systems," *IEEE Access*, vol. 4, pp. 4946–4958, 2016.
[28] A. Qasim, S. A. R. Kazmi, and I. Fakhir, "Formal specification and verification of real-time multi-agent system using timed arc Petri nets," *Adv. Elect. Comput. Eng.*, vol. 15, no. 3, pp. 73–78, 2015.
[29] A. Qasim, S. A. R. Kazmi, and I. Fakhir, "Executable semantics for the formal specification and verification of E-agents," *Indian J. Sci. Technol.*, vol. 8, no. 16, p. 1, 2015.
[30] B. Bartels and M. Kleine, "A CSP-based framework for the specification, verification, and implementation of adaptive systems," in *Proc. 6th Int. Symp. Softw. Eng. Adapt. Self-Manage. Syst.*, 2011, pp. 158–167.
[31] D. B. Abeywickrama and F. Zambonelli, "Model checking goal-oriented requirements for self-adaptive systems," in *Proc. IEEE 19th Int. Conf. Workshops Eng. Comput.-Based Syst. (ECBS)*, Apr. 2012, pp. 33–42.
[32] D. B. Abeywickrama, N. Hoch, and F. Zambonelli, "SimSOTA: Engineering and simulating feedback loops for self-adaptive systems," in *Proc. Int. C Conf. Comput. Sci. Softw. Eng.*, 2013, pp. 67–76.
[33] B. Djoudi, C. Bouanaka, and N. Zeghib, "Model checking pervasive context-aware systems," in *Proc. IEEE 23rd Int. WETICE Conf.*, Jun. 2014, pp. 92–97.
[34] R. Bruni, A. Corradini, F. Gadducci, A. L. Lafuente, and A. Vandin, "Modelling and analyzing adaptive self-assembly strategies with maude," *Sci. Comput. Program.*, vol. 99, pp. 75–94, Mar. 2015.
[35] E. Riccobene and P. Scandurra, "Towards ASM-based formal specification of self-adaptive systems," in *Proc. Int. Conf. Abstr. State Mach., Alloy, B, TLA, VDM, Z*. Berlin, Germany: Springer, 2014, pp. 204–209.
[36] A. K. Nabih, M. M. Gomaa, H. S. Osman, and G. M. Aly, "Modeling, simulation, and control of smart homes using Petri nets," *Int. J. Smart Home*, vol. 5, no. 3, pp. 1–14, 2011.



**MUHAMMAD ILYAS FAKHIR** received the M.Sc. degree in computer science from Iqra University, Karachi, Pakistan, in 2002, and the M.S. degree in computer science from the University of Central Punjab, Lahore, in 2012. He is currently pursuing the Ph.D. degree in computer science with the Computer Science Department, Government College University. In 2008, he joined the Computer Science Department, Government College University, as a Lecturer, where he is currently an Assistant Professor. He has authored seven research papers in peer-reviewed ISI indexed journals. His research interests include Petri nets, multi-agent systems, concurrent systems, and self-adaptive systems.

**SYED ASAD RAZA KAZMI** received the M.S. degree in computer science from Sir Syed Engineering University, Karachi, Pakistan, in 2004, and the Ph.D. degree in computer software and theory from the State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences, Beijing, China, in 2008. He is currently an Assistant Professor/In-charge with the Department of Computer Science, Government College University, Lahore. He conducts research in the area of formal methods specifically using techniques like LTL, CTL, and modal mu-calculus to specify the properties of reactive and concurrent systems for the verification of properties like safety, liveness, and deadlock. He also focused on the area of formal modeling of real-time multi-agent systems, model checking, compositional reasoning, and fixed point theory. He has authored or co-authored over 15 peer-reviewed ISI indexed journals.